RELAZIONE DEL PROGETTO JTRASH

Il progetto scelto è quello del gioco di carte Trash ed è stato strutturato con il pattern MVC (model, control, view) che permette all'utente di interagire con il gioco attraverso un'interfaccia grafica implementata nella view tramite Java Swing, senza però vedere tutti i dettagli implementativi presenti nel model.

Il gioco è suddiviso in tre panel, visualizzabili tramite tre bottoni posti in fondo all'interfaccia grafica. Per accedere al Profilo dell'utente, sarà necessario immettere tramite input una stringa di testo contenente il nickname, che verrà poi visualizzato in cima al pannello e salvato all'interno di un file che contiene sia il numero di partite vinte sia il numero di partite perse dal giocatore, aggiornati poi a fine partita.
L'avatar è visualizzabile in alto a sinistra nel profilo.
Dopo aver inserito il numero di giocatori che parteciperà, è possibile cliccare su "Gioca" e iniziare. Il controller si occuperà di utilizzare tutti gli elementi creati nel model per gestire la partita e mostrare all'utente solo l'interfaccia di gioco.

Nel **modello** sono presenti nove classi che rappresentano tutti gli elementi che contraddistinguono ogni partita:

- una classe astratta "Giocatori" che racchiude tutti i metodi utili ai giocatori come pescare una carta da tenere in mano per scambiarla con quelle del nostro mazzo, scartare una carta o restituire l'ArrayList della mano. Si è preferito usare una classe astratta invece di una interfaccia per avere un legame tra le sottoclassi più forte e non per sottolineare un semplice comportamento comune
- ci saranno "Umano" e "Bot" come sottoclassi, che erediteranno tutti gli attributi e metodi di "Giocatori" che devono essere implementati. Mentre il Bot "giocherà da solo" e quindi avrà un metodo che permette di pescare una carta, controllare non sia uno scarto e sostituirla ad una della propria mano fino a trovare una carta da scartare, il Giocatore umano utilizzerà l'interfaccia grafica per interagire col gioco
- una classe astratta "Carta" servirà a raccogliere tutti gli attributi e metodi eseguibili dalle due tipologie di carte che avremo: carte con un valore numerico e carte con una figura. Le prime sono rappresentate da un numero e un seme, mentre le seconde hanno quattro tipi di figure che o permettono di posizionare la

carta dove si vuole o di scartarla subito. Ovviamente una volta che si scarta, si passa al giocatore successivo

- ci sono due mazzi di carte utilizzati: quelli da cui si pesca e quelli dove si mettono gli scarti. Entrambi gli oggetti sono rappresentati da uno stack di carte, strutture dati utili per prendere sempre la carta in cima o per posizionarne una infondo
- infine si avrà una classe “RegoleGioco” che raccoglie appunto tutto il regolamento del gioco. Determina quando una carta è da scartare, come scambiare la carta che si ha in mano con una del proprio mazzo, come far andare avanti i turni ed infine stabiliscono quando un round o l’intera partita è finita. Per controllare che un round sia terminato e che quindi tutte le carte nella mano di un giocatore siano state girate, si utilizza uno *stream* che prende la mano e la filtra in base a quante carte siano girate, restituendone un nuovo array da cui si possono contare le carte filtrate. Questa classe eredita metodi dall’Observable, perché saranno gli observer (giocatori) a guardare l’andamento della partita e questa classe li notificherà ogni qual volta ci sarà un cambio di turno (oltre a rimuovere o aggiungere nuovi observer).

Per quanto riguarda la **GUI** del programma, essa interagisce con il controller attraverso un frame principale che racchiude tre diversi panel dove sono visualizzabili i tre stage del programma. Infatti inizialmente l’utente si troverà davanti al menù, poi potrà crearsi il profilo con un nome utente e infine giocare effettivamente una partita.
Il frame centrale implementa il metodo dell’interfaccia Observer definendo il comportamento da avere a seguito del cambio di stato dell'oggetto osservato. Nella classe Central Frame è presente il main e gestisce la navigazione tra i panel.
Attraverso un JOptionPane richiede in input all’utente il suo nome (scrivendo subito nel file dei punteggi) e quanti giocatori saranno presenti, per poi rendere possibile il click su “Gioca”.
Ci sono anche metodi per mostrare a schermo che siamo nel turno del giocatore umano (con un message dialog) ed evidenziare ogni volta l’utente che può effettivamente giocare in quel momento. Qui ci sarà anche un metodo che serve ad aggiungere una carta data in input dal controller sotto forma di stringa, che poi dovrà essere visualizzata nella view.
Inoltre è anche presente un metodo da chiamare quando finisce la partita (ovvero quando un giocatore possiede una sola carta ed essa viene girata): se l’umano ha vinto, verranno aggiornate le sue vittorie nel file, altrimenti verrà aggiornato il punteggio delle sconfitte.
Ogni accesso per scrittura o lettura al file dei punteggi deve essere controllato e gestito da un blocco try-catch.

Per quanto riguarda la musica di sottofondo ho utilizzato Audio Input Stream per leggere un file mp3 e riprodurlo in loop fino alla chiusura dell'applicazione. C'è anche un suono per ogni click sui tasti.
Nel pannello di gioco, utilizzeremo la struttura switch-case per differenziare la visualizzazione dei mazzi dei giocatori in base al loro numero (potremmo avere 1, 2 o 3 bot). I metodi repaint() e revalidate(); servono ad aggiornare la view con nuovi elementi grafici eliminando i precedenti.

Nel package **controller** troviamo la classe "Game Controller" che prendendo come input la view, gestirà l'intero gioco. Nel costruttore si crea un array di giocatori in base al numero fornito in input dall'utente e poi con questo si crea un oggetto "Regole Gioco" che inizializza i due mazzi utilizzati e distribuisce dieci carte ad ogni giocatore.
Si usa un metodo per controllare che la carta che si ha in mano sia valida e possa essere giocata, altrimenti lo comunicheremo attraverso la view all'utente. Col metodo "avanzaTurno" invece, faremo procedere la partita controllando che il giocatore corrente sia umano o un bot. In questo ultimo caso, metteremo in cima alla pila degli scarti la carta restituita dal suo metodo "gioca" e aggiorneremo anche la vista. Successivamente verrà chiamato il metodo "passa" che si occupa effettivamente di aggiornare il giocatore successivo e notificarlo a tutti i players, oltre ad evidenziarne il nome.
Per poter visualizzare gli scarti dei bot è necessario fermare l'esecuzione dei metodi tra un turno e un altro. Per fare questo si usa una classe "Timer" con un unico metodo "run" che permette di fermare per 3 secondi quell'esatto thread che è in esecuzione, per poi procedere con "avanzaTurno".